

Outils pour la synchronisation

TD2 : Les threads POSIX

L'objectif de ce TP est de synchroniser plusieurs threads à l'aide de mutex et de variables conditionnelles.

Pour ce TP, on vous fournit une librairie permettant de gérer l'affichage de rectangles dans une fenêtre. Allez regarder barx.h afin de consulter la liste des fonctions disponibles.

Exercice 1 : Prise en main de la librairie barx

Question 1 : Le fichier bar.c vous montre comment utiliser les fonctions de la librairie. Compiler, linker et exécuter le programme bar.c en utilisant la ligne de commande suivante :

```
gcc bar.c -o bar -L. -lbarx -lX11 -lpthread
```

Exercice 2 : Multithreading

Question 1 : Ecrivez un programme thx.c ayant le comportement suivant :

Le thread principal (main) :

- réalise les initialisations
- crée 3 threads exécutant la fonction th_fonc
- attend la fin des threads

La fonction th_fonc :

- si thread numéro i :
 - écrire le numéro dans la fenêtre
 - tracer un rectangle vide
 - boucle de longueur mi :
 - printf message
 - usleep
 - faire progresser le remplissage du rectangle
- à la fin du thread i :
 - retourner son numero

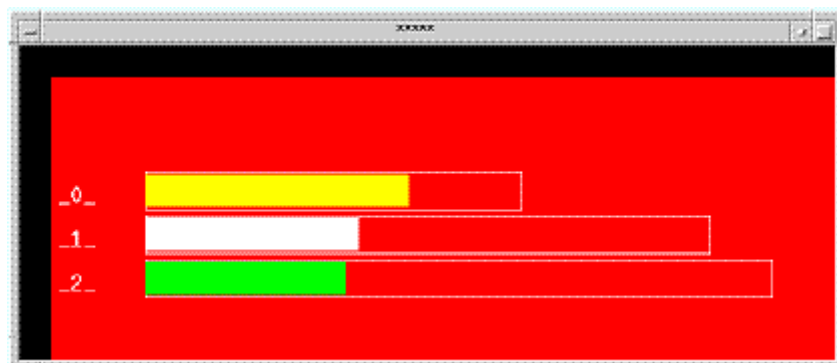


Figure 1: Illustration du comportement attendu de thx.c

Exercice 3 : Rendez-vous

Question 1 : Nous allons faire en sorte que les 3 threads se donnent rendez-vous quand chacun aura fait progresser son rectangle de 10 fois.

Ecrivez un programme `thrdv.c` se comportant comme `thx.c`, et auquel vous ajoutez :

dans la fonction `th_fonc` :

- si on a déjà fait progresser le rectangle de couleur du thread de 10 fois, alors appeler une fonction `rdv()`;
- comme sur la copie d'écran ci-dessous, on ajoutera sur le dessin une marque verticale à l'endroit du rendez-vous.

fonction `rdv()` :

- si le thread appelant n'est PAS le dernier qui arrive, alors il s'endort,
- si c'est le dernier arrivé alors il provoque le réveil de tous les autres et il continue.

Commencez par faire une version avec `pthread_barrier_t`, puis une version avec `pthread_cond_broadcast` et enfin une version avec `pthread_cond_signal`.

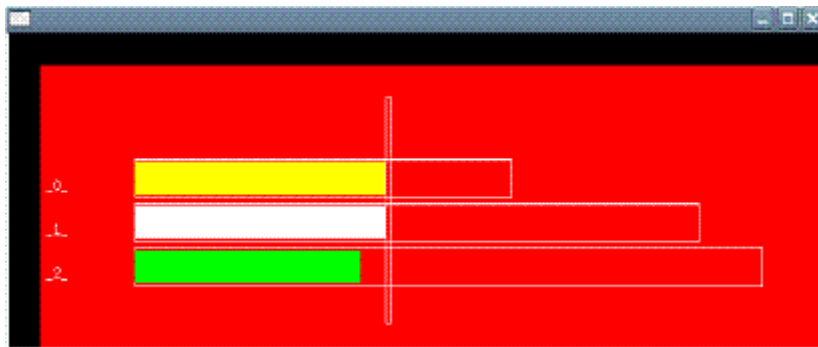


Figure 2: Illustration du comportement attendu de `thrdv.c` (les threads 0 et 1 sont bloqués au point de rendez-vous dans l'attente du dernier thread).

Question 2 : Vérifiez que votre barrière fonctionne si elle est utilisée deux fois de suite. Si ce n'est pas le cas, modifiez votre programme.

Question 3 : Faites une nouvelle version de votre programme en gérant le rendez-vous avec des sémaphores (utilisez la librairie `semaphore.h`).